

Messenger → WhatsApp Forwarder

Technical Diagram Set: architecture, UML, data-flow, business-process and algorithm diagrams

A self-hosted Node.js service that watches a Facebook Page inbox around the clock and forwards every incoming Messenger message (sender name, text, date and time, and attachments) to the business owner's WhatsApp in real time, using Meta's Messenger Platform webhooks and the WhatsApp Business Cloud API.

WHAT'S INSIDE



System Overview

Figures 1–4



Structural Design

Figures 5–7



Data Flow

Figure 8



Business Process

Figures 9–10



Process Flowcharts

Figures 11–18



Behavioural Models

Figures 19–24

NOTATIONS USED

- **UML 2.5:** use case, deployment, component, class, sequence and state machine diagrams
- **Data flow diagrams:** context (Level 0) and Level 1 in Gane & Sarson notation
- **Flowcharts:** standard flowchart symbols (ISO 5807 style), cross-functional swimlanes and a phased process map

TRACEABILITY

Every diagram was traced line by line against the project source code (Node.js, Express 4, dotenv, Docker) and then independently re-checked. Function names, routes, limits, retry delays and error codes match the implementation exactly. Each page cites the files it was drawn from.

List of figures

FIG.	TITLE	DIAGRAM TYPE	PAGE
A • System Overview			
1	System Context Diagram	Data Flow Diagram • Level 0 (context)	4
2	System Architecture	Architecture • layered block diagram	5
3	Use Case Diagram	UML 2.5 • Use case	6
4	Deployment Diagram	UML 2.5 • Deployment	7
B • Structural Design			
5	Component Diagram	UML 2.5 • Component	8
6	Module Structure Diagram	UML 2.5 • Class notation for modules	9
7	Data Structure Diagram	UML 2.5 • Class (data model)	10
C • Data Flow			
8	Data Flow Diagram — Level 1	DFD • Gane & Sarson • Level 1	11
D • Business Process			
9	Customer Message Notification Process	Cross-functional (swimlane) flowchart	12
10	Setup and Go-Live Process	Process map • phased flowchart	13
E • Process Flowcharts			
11	Webhook Request Handling	Flowchart	14
12	Message Forwarding Logic (Part 1 of 2)	Flowchart	15
12	Message Forwarding Logic (Part 2 of 2)	Flowchart	16
13	Attachment Forwarding with Fallbacks	Flowchart	17
14	WhatsApp Delivery: Retry and 24-Hour Window Fallback	Flowchart	18
15	Webhook Signature Verification	Flowchart	19
16	Sender Name Resolution with Cache	Flowchart	20
17	Application Start-up and Configuration	Flowchart	21
18	Credential Test Script	Flowchart	22
F • Behavioural Models			
19	Sequence: Webhook Verification Handshake	UML 2.5 • Sequence	23
20	Sequence: Text Message Forwarding	UML 2.5 • Sequence	24
21	Sequence: Attachment Forwarding with Fallback	UML 2.5 • Sequence	25
22	Sequence: 24-Hour Window and Template Fallback	UML 2.5 • Sequence	26
23	State Machine: Webhook Event Lifecycle	UML 2.5 • State machine	27
24	State Machine: Outbound WhatsApp Message	UML 2.5 • State machine	28

The system at a glance

What it does

Receives Facebook Page Messenger events by webhook, verifies Meta's HMAC signature, looks up the sender's name, and sends a formatted notification (name, date/time, text and attachments) to one WhatsApp number.

How it stays reliable

Acknowledges Meta at once, drops duplicates and echoes, retries transient WhatsApp errors (2 s, 4 s, 8 s), falls back to an approved template when a send is rejected because the 24-hour window is closed, and re-tries failed attachments by upload and then as a plain link.

Technology

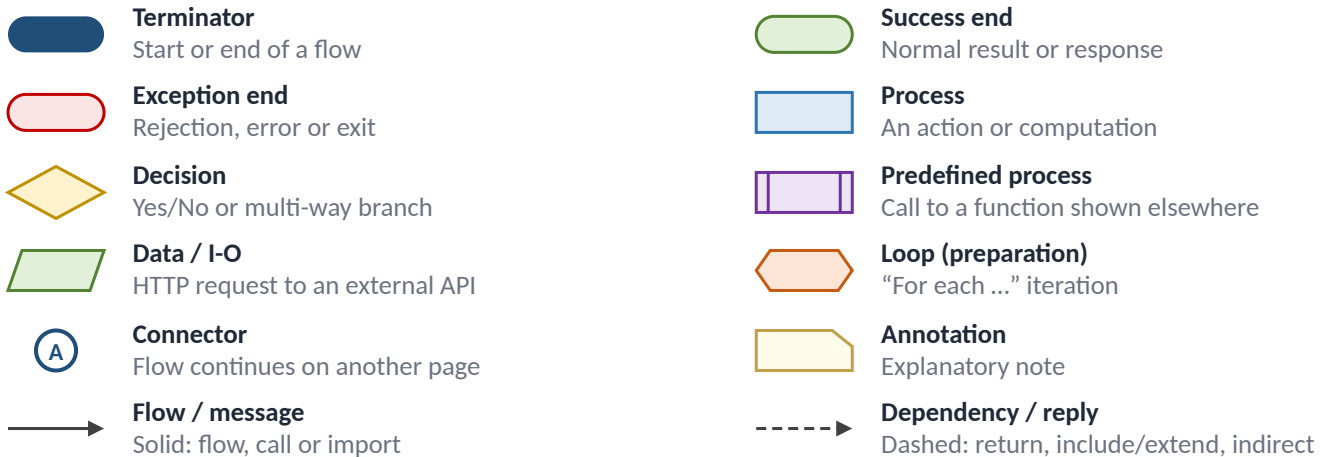
Node.js ≥ 18 (ES modules) · Express 4 · dotenv · Meta Graph API v21.0 (default) · WhatsApp Business Cloud API · Docker (node:20-alpine) · node:test for unit tests.

Source files

```
src/server.js      src/verifySignature.js
src/forwarder.js   src/messenger.js
src/whatsapp.js    src/config.js
scripts/check-whatsapp.js
test/verifySignature.test.js
```

How to read the flowcharts

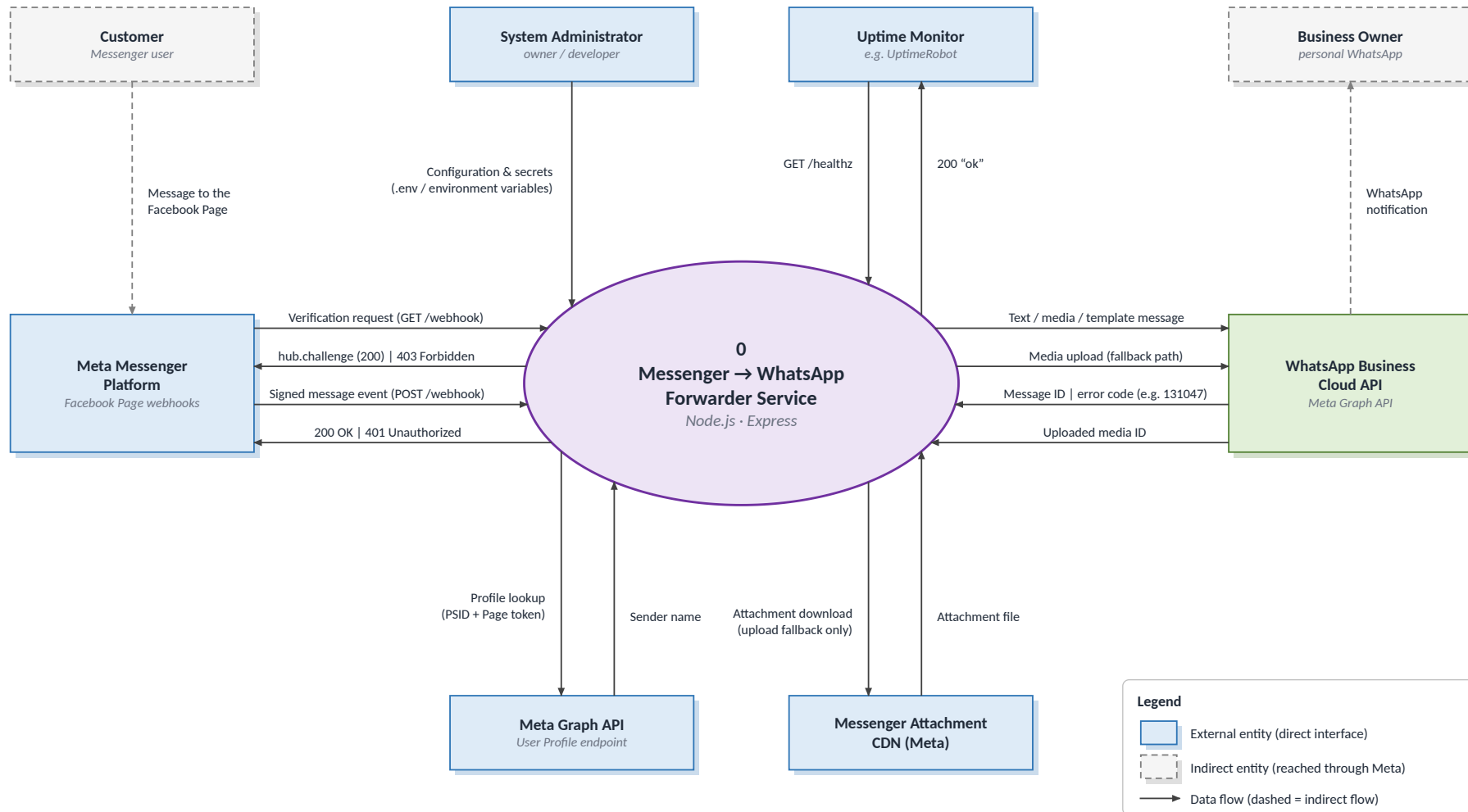
Every flowchart uses the same shapes and colours. UML and DFD figures follow their standard notations and include their own legends where needed.



Terms used

PSID	Page-scoped user ID: Meta's identifier for a customer within one Facebook Page.
mid	Unique Messenger message ID, used to detect redelivered events.
24-hour window	WhatsApp accepts free-form business messages only within 24 hours of the recipient's last message; outside it a send fails with error 131047.
Echo	A webhook event for a message the Page itself sent; ignored to avoid loops.

FIGURE 1
System Context Diagram



- The whole service is one process (0). Every arrow is a data flow that exists in the code; dashed flows happen outside the service, inside Meta's platform.
- Meta's Messenger Platform calls the service in two ways: a one-time verification request (GET), and signed POSTs that each carry one or more events in `entry[].messaging[]`.
- The attachment CDN is contacted only when sending media by link fails (the upload fallback).

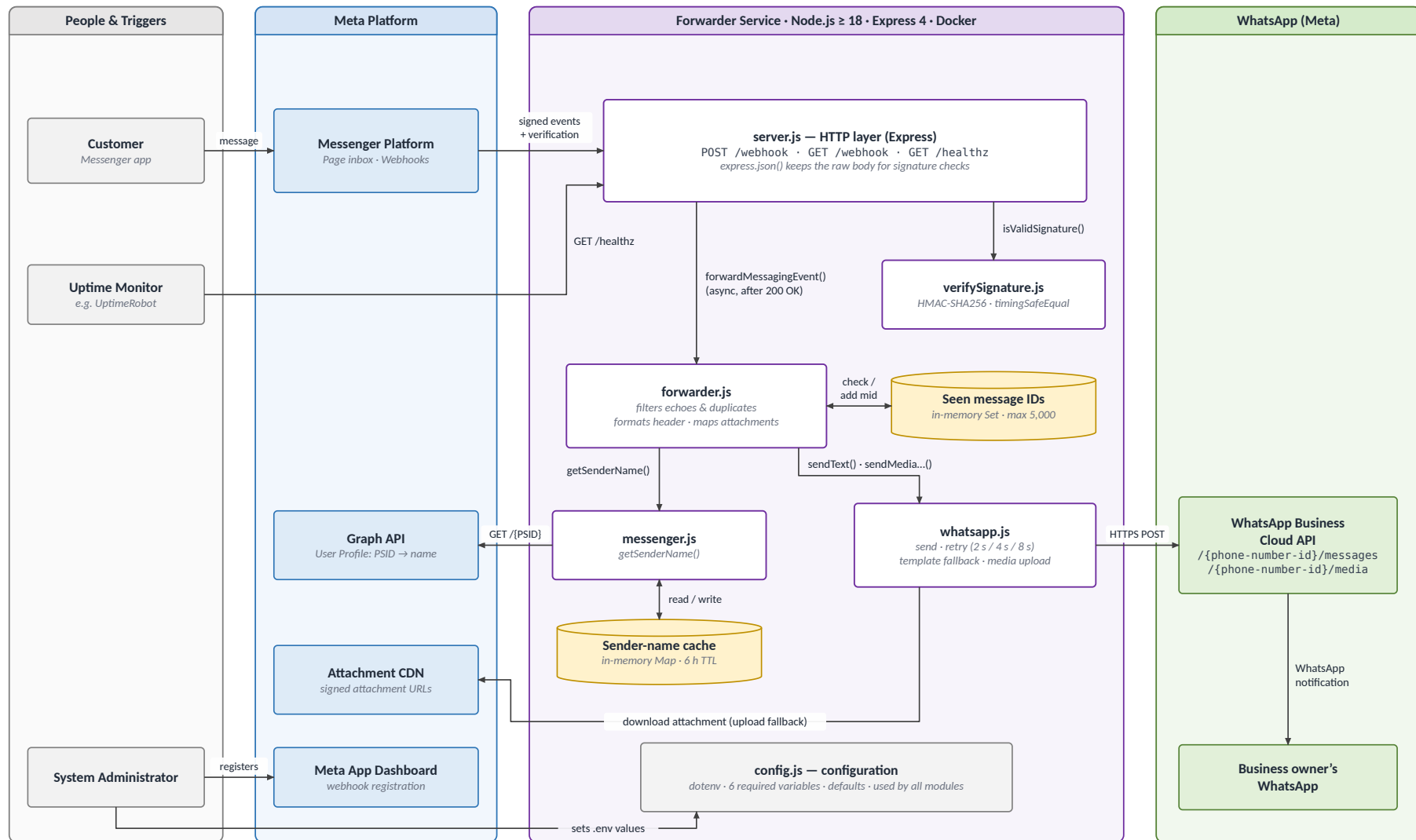
SOURCE `src/server.js` • `src/messenger.js` • `src/whatsapp.js`

FIGURE 2

System Architecture

Architecture · layered block diagram

A · System Overview

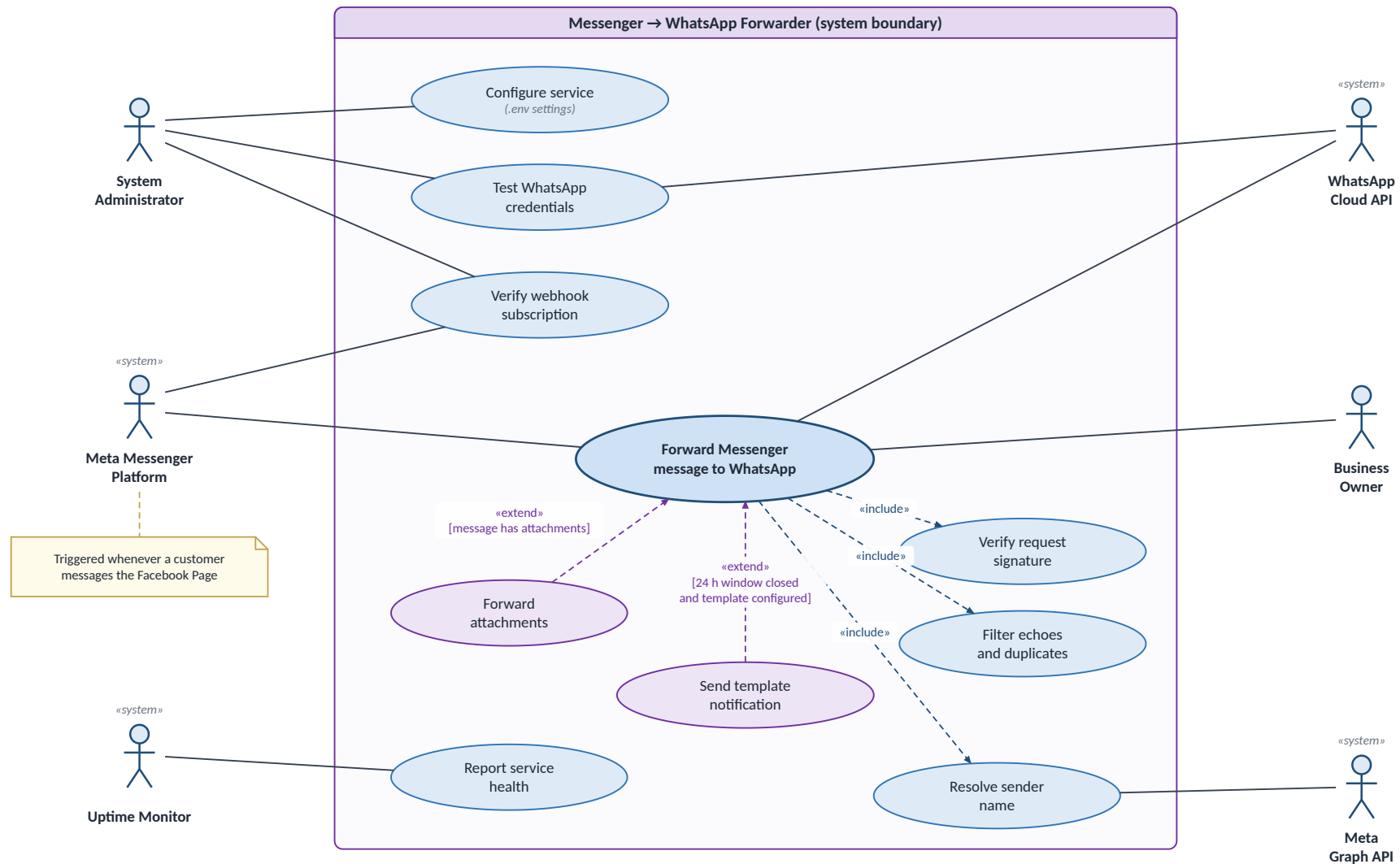


- Six ES modules. `server.js` is the only entry point of the running service; `check-whatapp.js` is a separate setup script (Figure 18).
- All state is in memory: a Set of seen message IDs (max 5,000) and a sender-name Map (6-hour TTL). Nothing is written to disk.
- The webhook is acknowledged with HTTP 200 before any processing, so slow WhatsApp calls never delay Meta.

SOURCE `src/*.js` · Dockerfile

FIGURE 3

Use Case Diagram

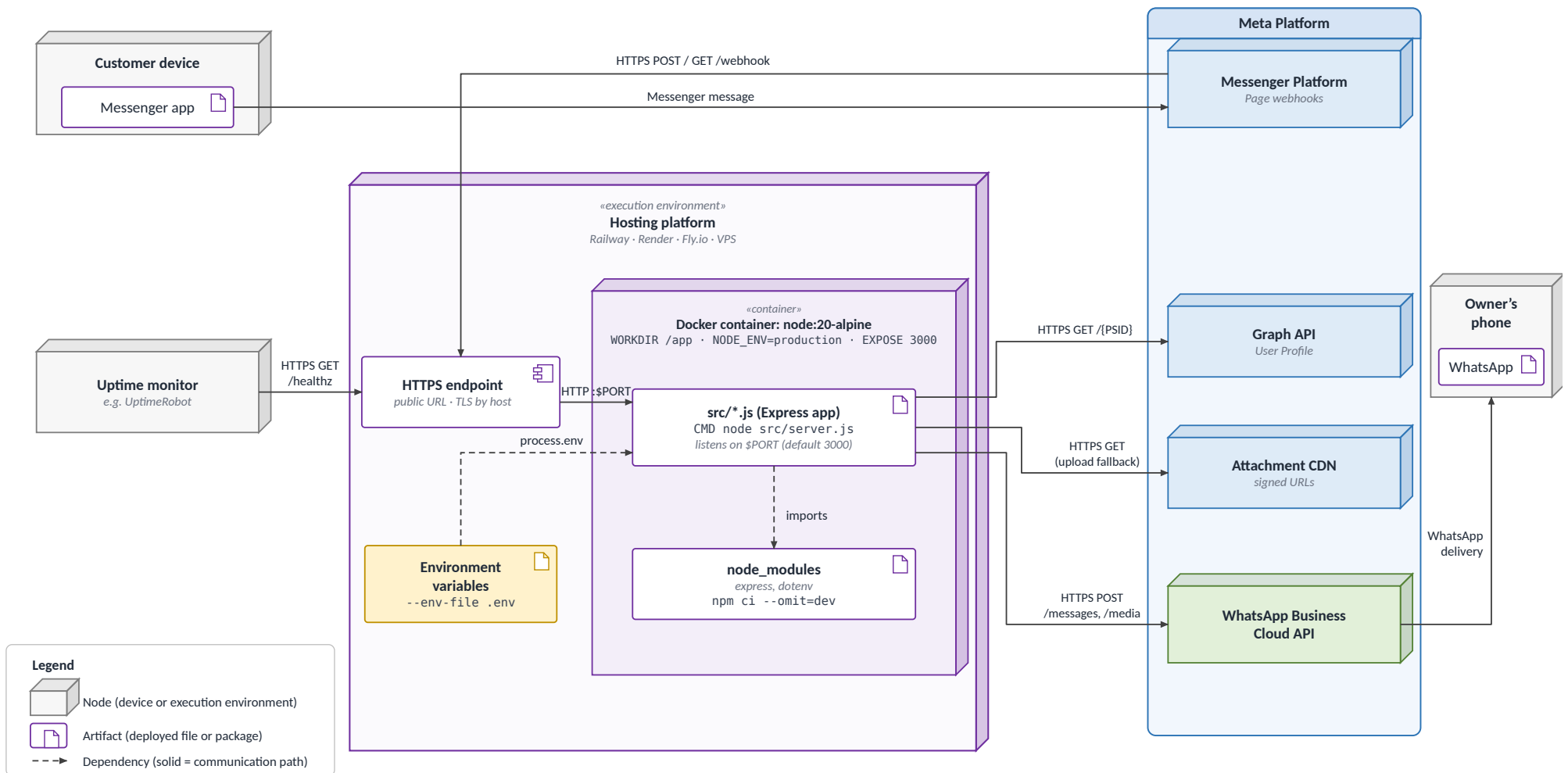


- Meta's Messenger Platform is the primary actor for forwarding because it delivers the webhook. The customer only interacts with Facebook, so is shown as a note.
- «include» marks behaviour that always runs as part of forwarding. «extend» marks optional behaviour that runs only under the condition shown.
- "Test WhatsApp credentials" is the npm run test:whatsapp script; "Report service health" is GET /healthz.

SOURCE src/server.js • src/forwarder.js • scripts/check-whatsapp.js

FIGURE 4

Deployment Diagram

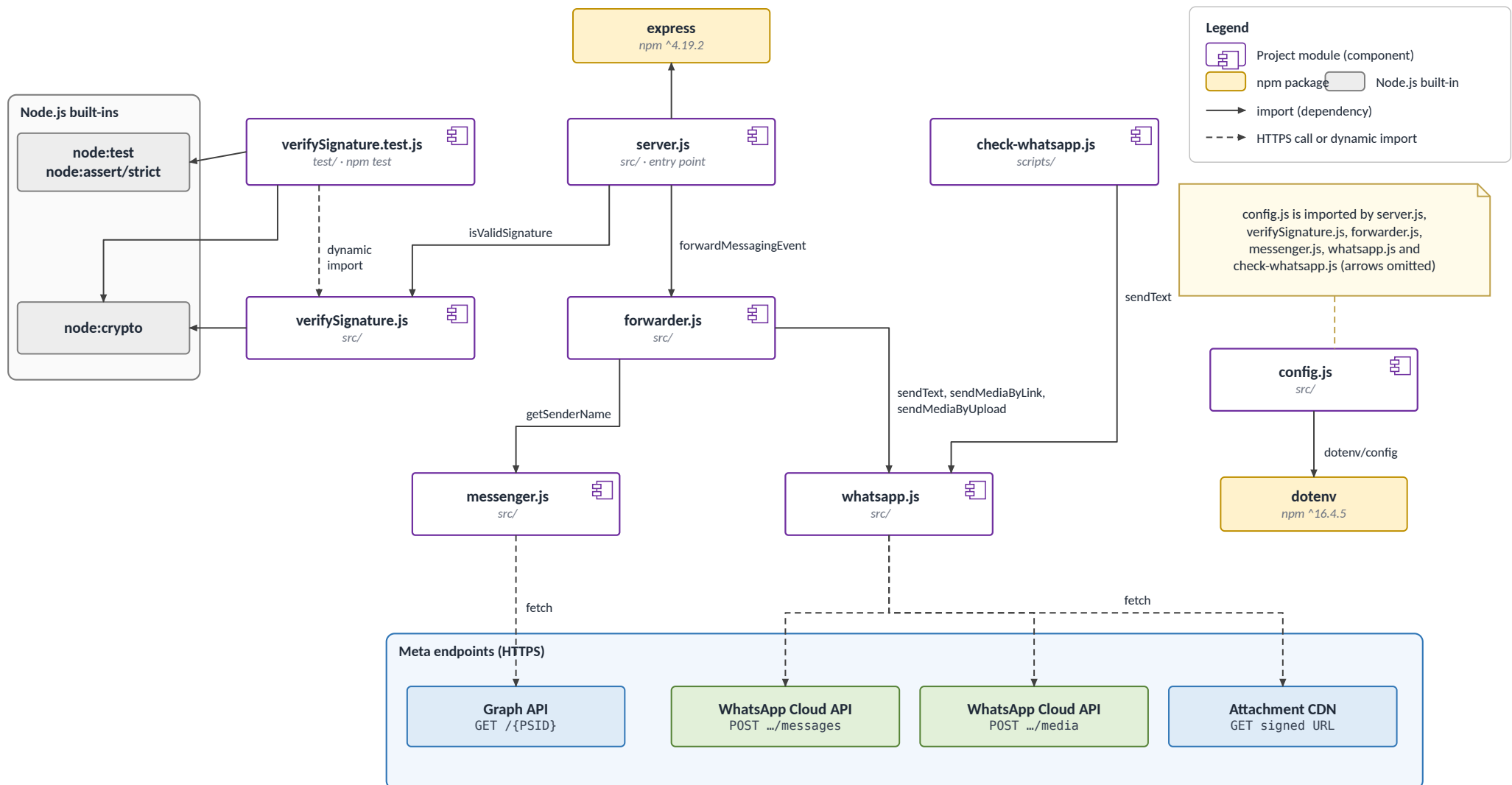


- The Dockerfile builds from node:20-alpine, installs production dependencies (npm ci --omit=dev), copies src/, exposes port 3000 and runs node src/server.js.
- Secrets are supplied as environment variables (docker run --env-file .env); the .env file is never copied into the image.
- Meta requires HTTPS for webhooks, so the hosting platform provides the public TLS endpoint (ngrok can do this for local testing).

SOURCE Dockerfile · README.md (Step 4)

FIGURE 5

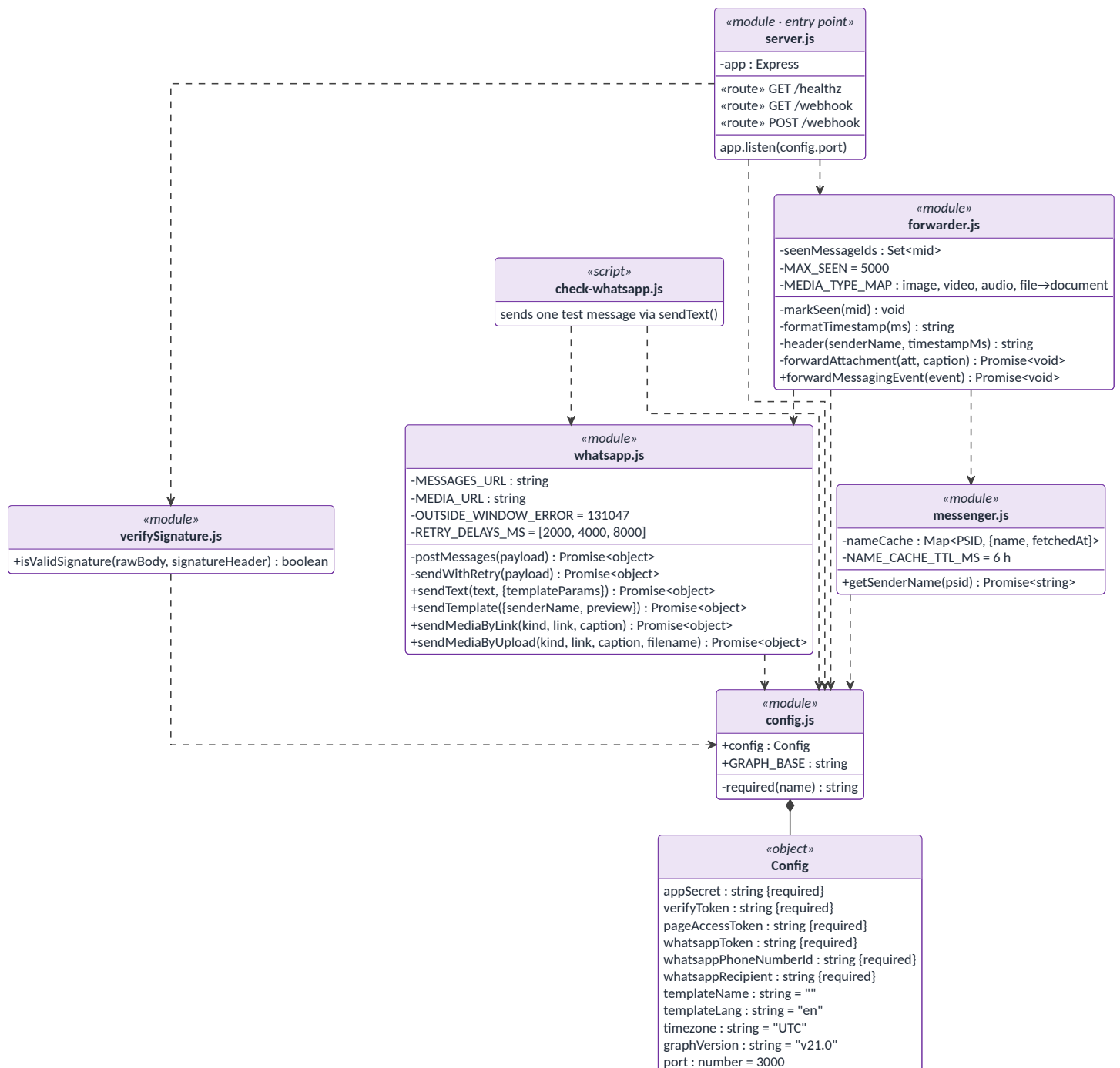
Component Diagram



- Solid arrows are ES module imports. Dashed arrows are HTTPS calls made with Node's global fetch, plus the unit test's dynamic import.
- The only npm dependencies are express (HTTP server) and dotenv (loads .env). Cryptography uses the built-in node:crypto.
- config.js is imported by six files; those arrows are omitted to keep the diagram readable.

SOURCE package.json · import statements in src/, scripts/ and test/

Module Structure Diagram

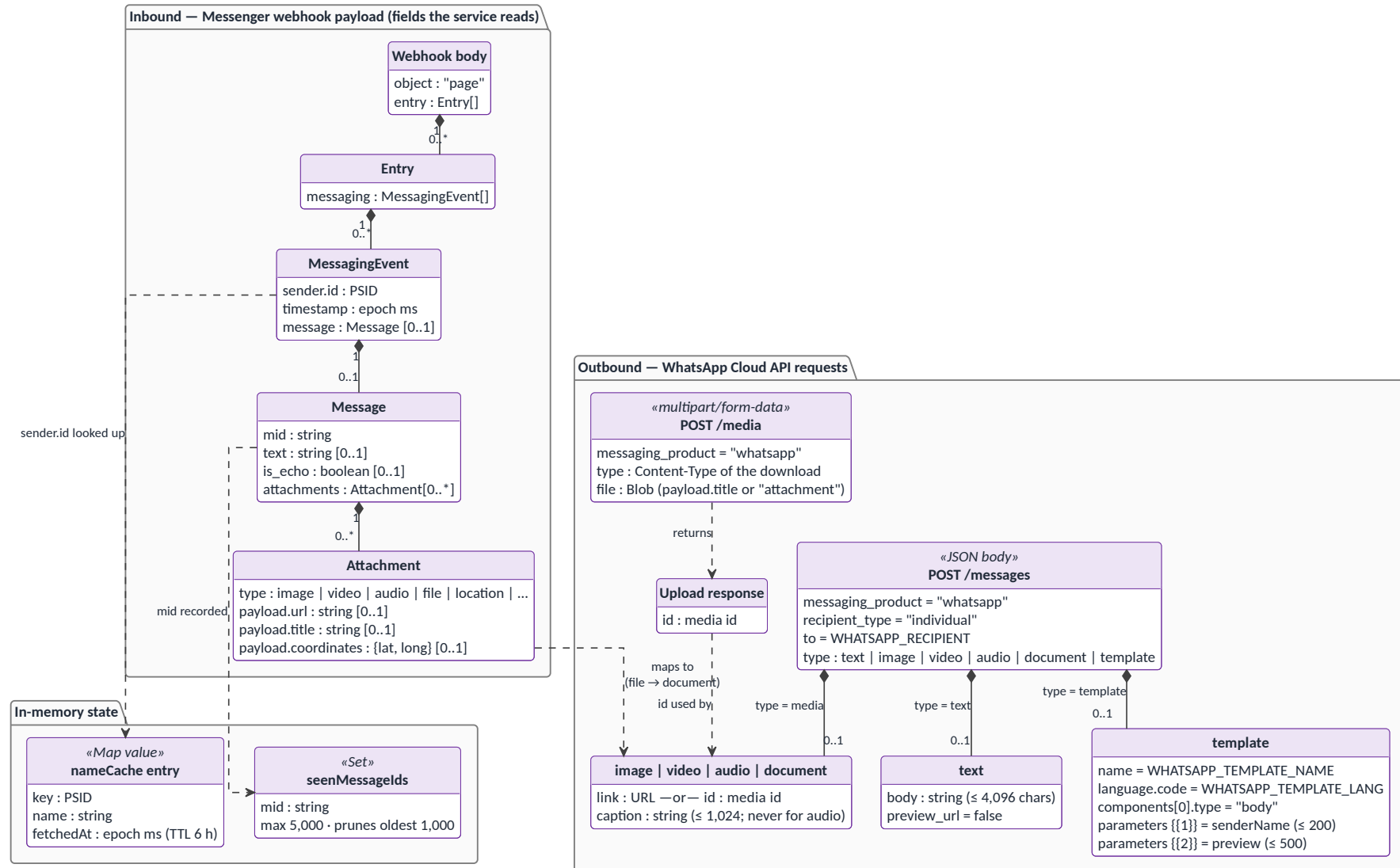


- Each JavaScript module is drawn as a class stereotyped «module»; dashed arrows are import dependencies. + marks exports, - module-private state and helpers.
- Config holds six required variables (the service exits at start-up if any is missing) and five optional ones with defaults.
- Constants are the real values: MAX_SEEN 5000, NAME_CACHE_TTL_MS 6 h, OUTSIDE_WINDOW_ERROR 131047, RETRY_DELAYS_MS [2000, 4000, 8000].

SOURCE src/*.js • scripts/check-whatsapp.js

FIGURE 7

Data Structure Diagram



- There is no database. This models the JSON received from Meta, the requests sent to the WhatsApp Cloud API, and the two in-memory stores.
- Only fields the code reads or writes are shown. Limits applied by the code: text 4,096 characters, captions 1,024, template parameters 200 and 500.
- Attachment mapping: image → image, video → video, audio (including voice messages) → audio, file → document.

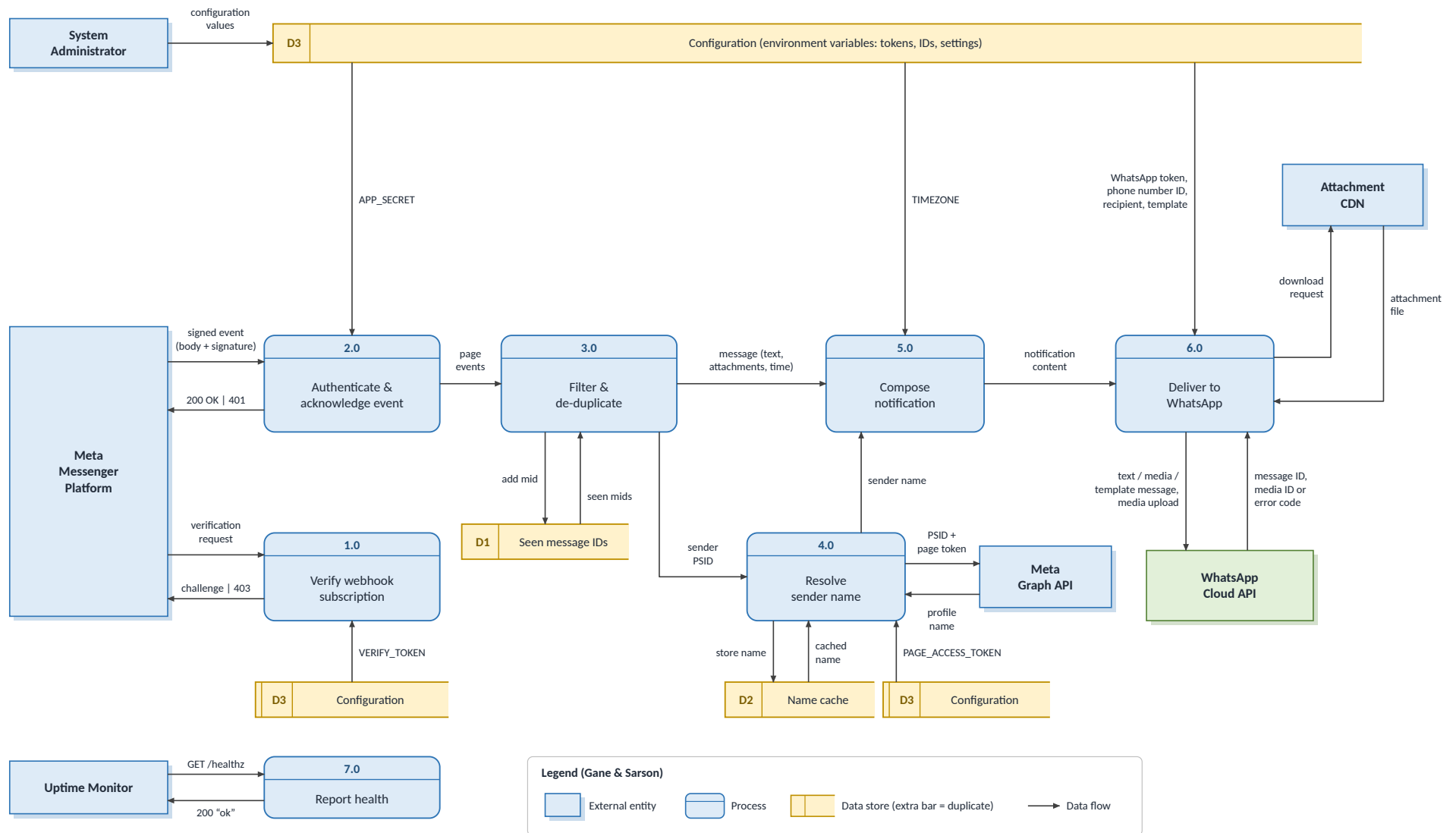
SOURCE src/forwarder.js · src/whatsapp.js · src/messenger.js

FIGURE 8

Data Flow Diagram — Level 1

DFD • Gane & Sarson • Level 1

C • Data Flow



- Breaks process 0 of Figure 1 into seven processes. D3 is drawn three times to avoid crossing lines; the two copies carry the duplicate-store bar.
- D1 and D2 are held in memory only, so they start empty whenever the service restarts. Process 7.0 is a health check and reads no data.

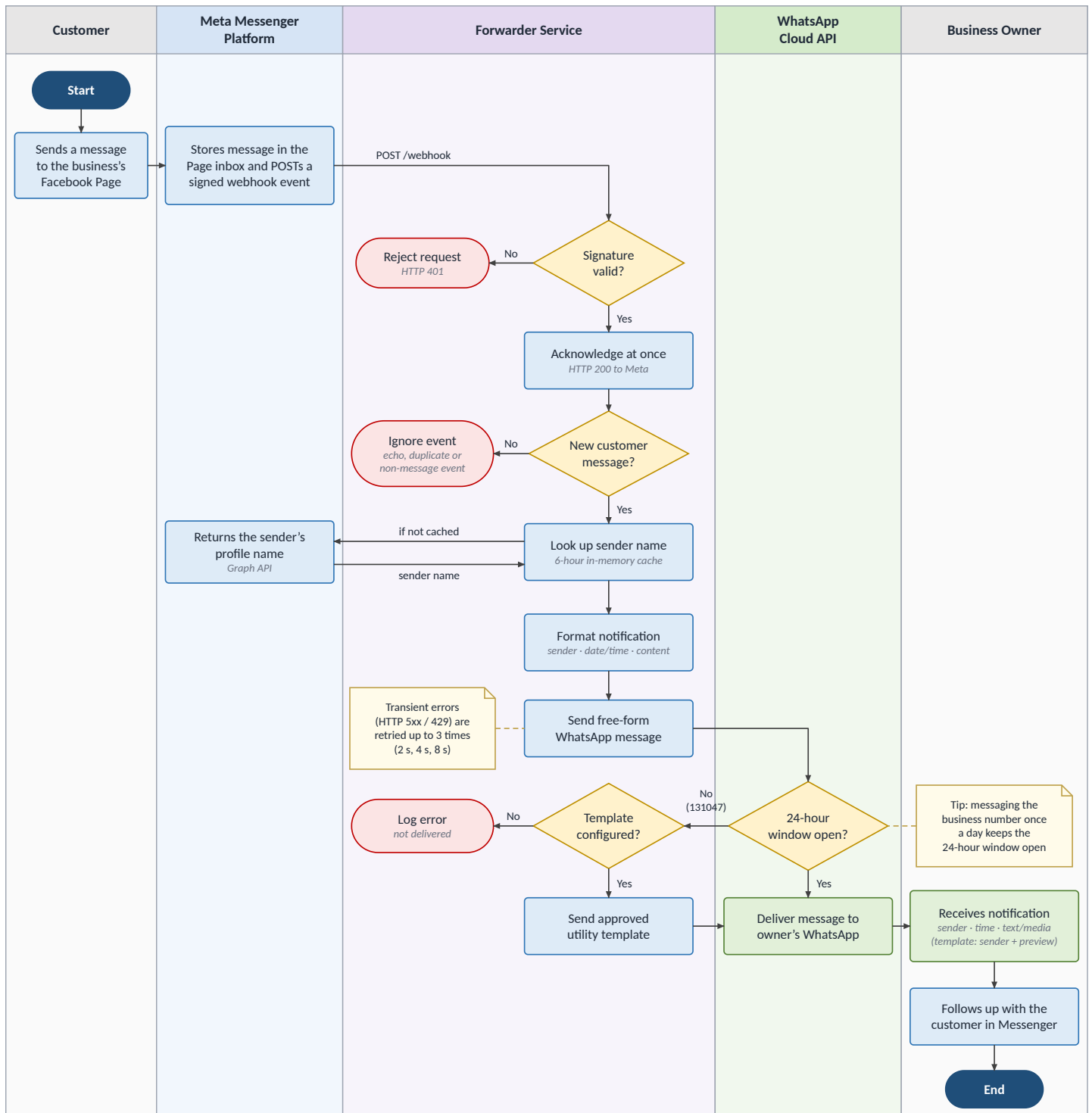
SOURCE src/server.js • src/forwarder.js • src/messenger.js • src/whatsapp.js • src/config.js

FIGURE 9

Customer Message Notification Process

Cross-functional (swimlane) flowchart

D · Business Process



- Shows who does what, end to end, when a customer messages the Facebook Page. Red terminators are the exception outcomes.
- If the Cloud API rejects the send with error 131047 (24-hour customer-service window closed), the service sends the pre-approved utility template, if one is configured.
- Following the README, the owner keeps the window open by messaging the business number once a day, so notifications can go out as free-form messages.

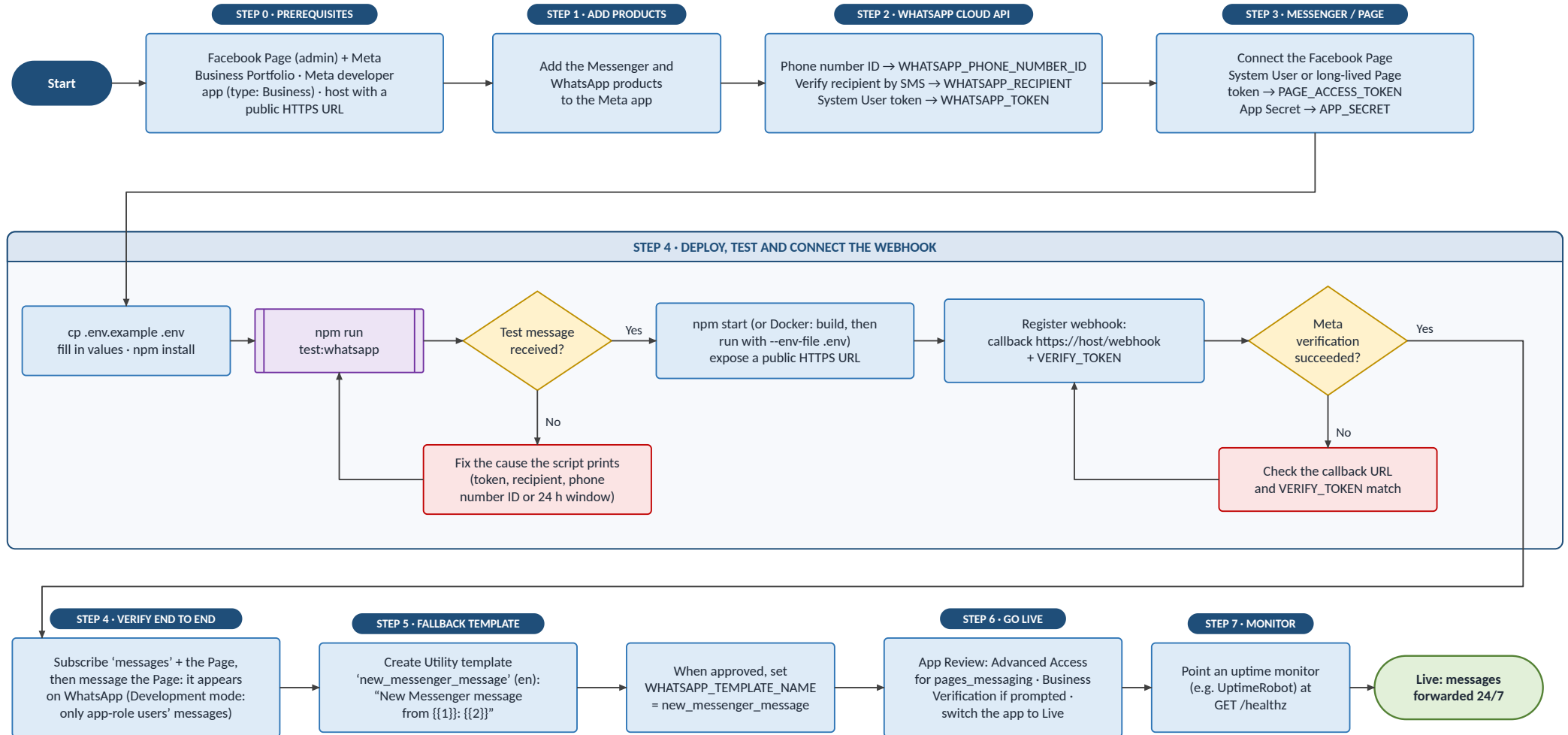
SOURCE src/server.js · src/forwarder.js · src/whatsapp.js · README.md

FIGURE 10

Setup and Go-Live Process

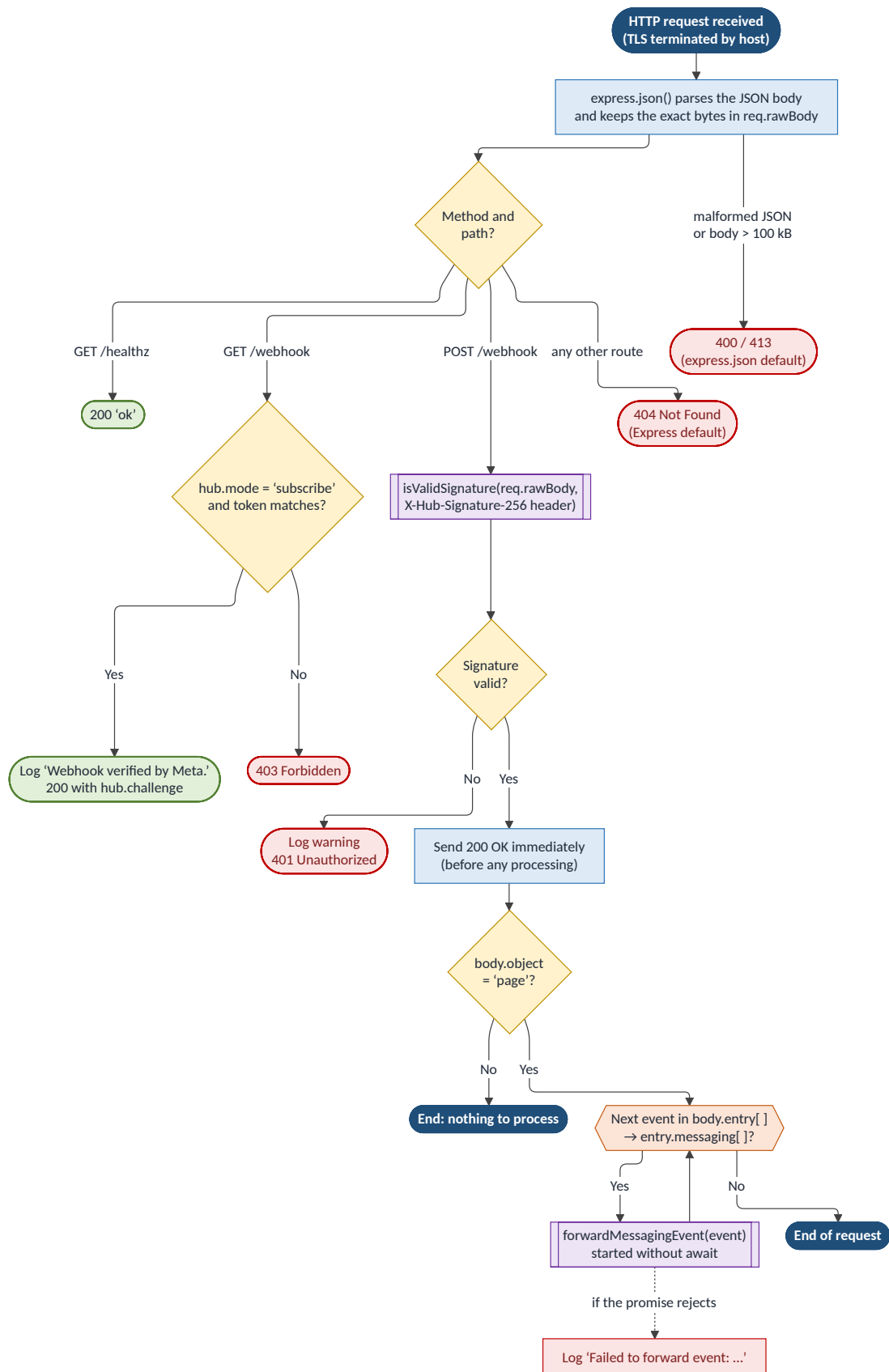
Process map · phased flowchart

D · Business Process



- Follows README Steps 0–7. The two loops are the built-in checks: the credential test script and Meta's webhook verification handshake.
- In Development mode only users with a role on the Meta app trigger webhooks. Public messages need Advanced Access to pages_messaging and Live mode (Step 6).

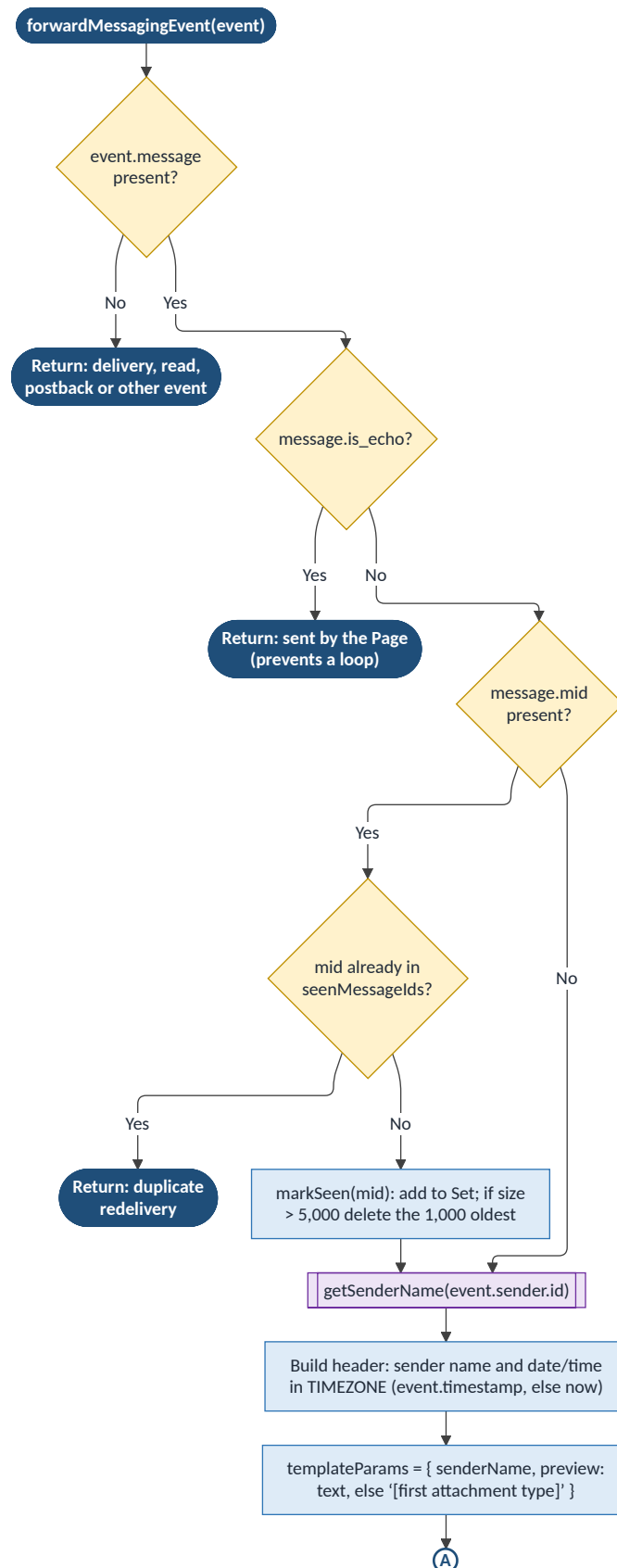
SOURCE README.md · scripts/check-whatsapp.js



- Verification succeeds only when `hub.mode` is 'subscribe' and `hub.verify_token` equals `VERIFY_TOKEN`; the response body is `hub.challenge`.
- For POST, 200 OK is sent before processing. Each messaging event goes to `forwardMessagingEvent` without `await`; a failure is only logged. The server speaks plain HTTP; the host terminates TLS.
- The signature is checked against the raw request bytes captured by `express.json({ verify })`, not re-serialised JSON.

SOURCE `src/server.js`

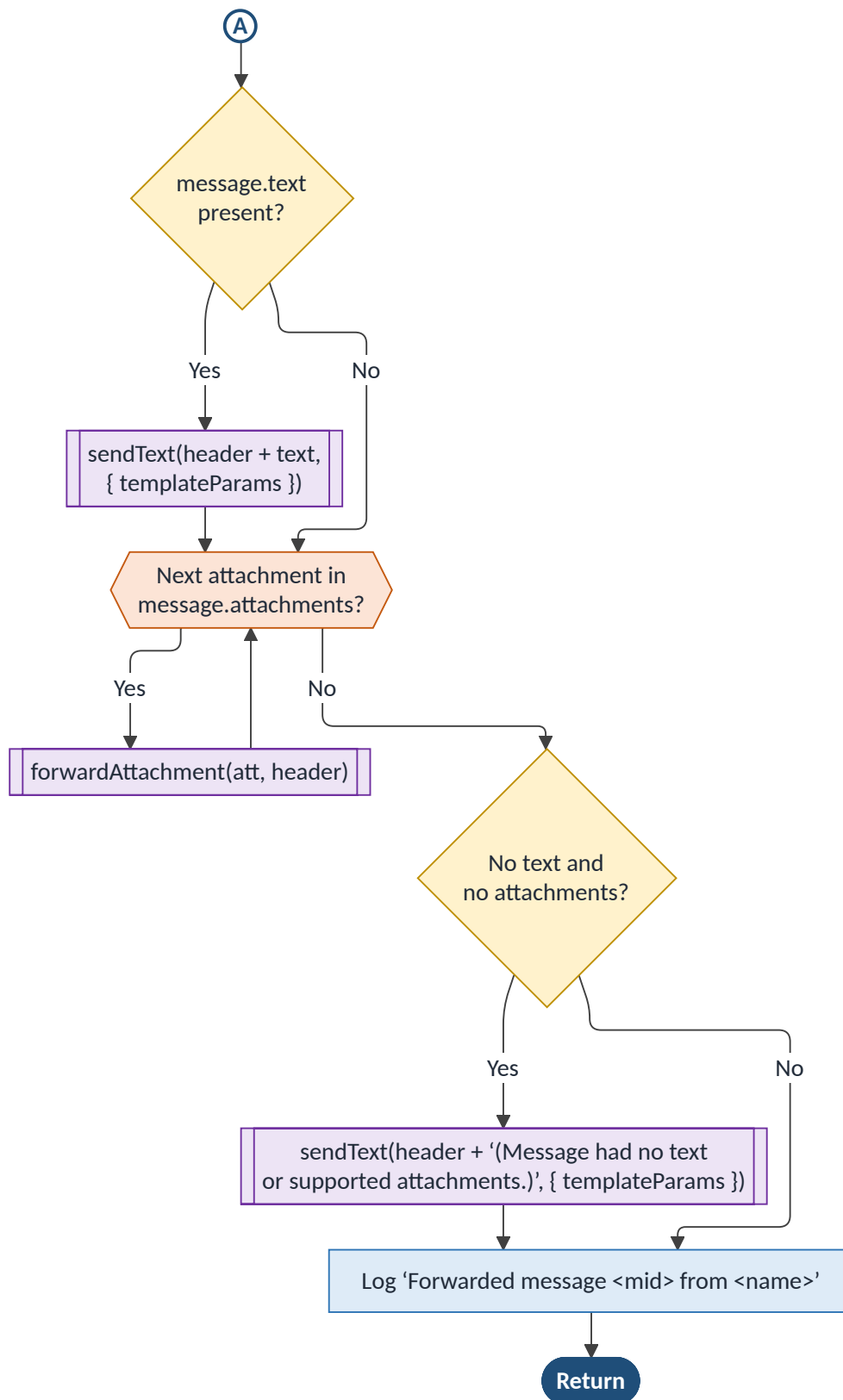
Message Forwarding Logic



- Guards run in order, dropping events with no message (delivery, read, postback), echoes of the Page's own replies, and redelivered duplicates.
- The seen-ID Set keeps at most 5,000 IDs. When it grows past that, the 1,000 oldest are deleted (a Set preserves insertion order).
- Connector A continues on the next page (Part 2).

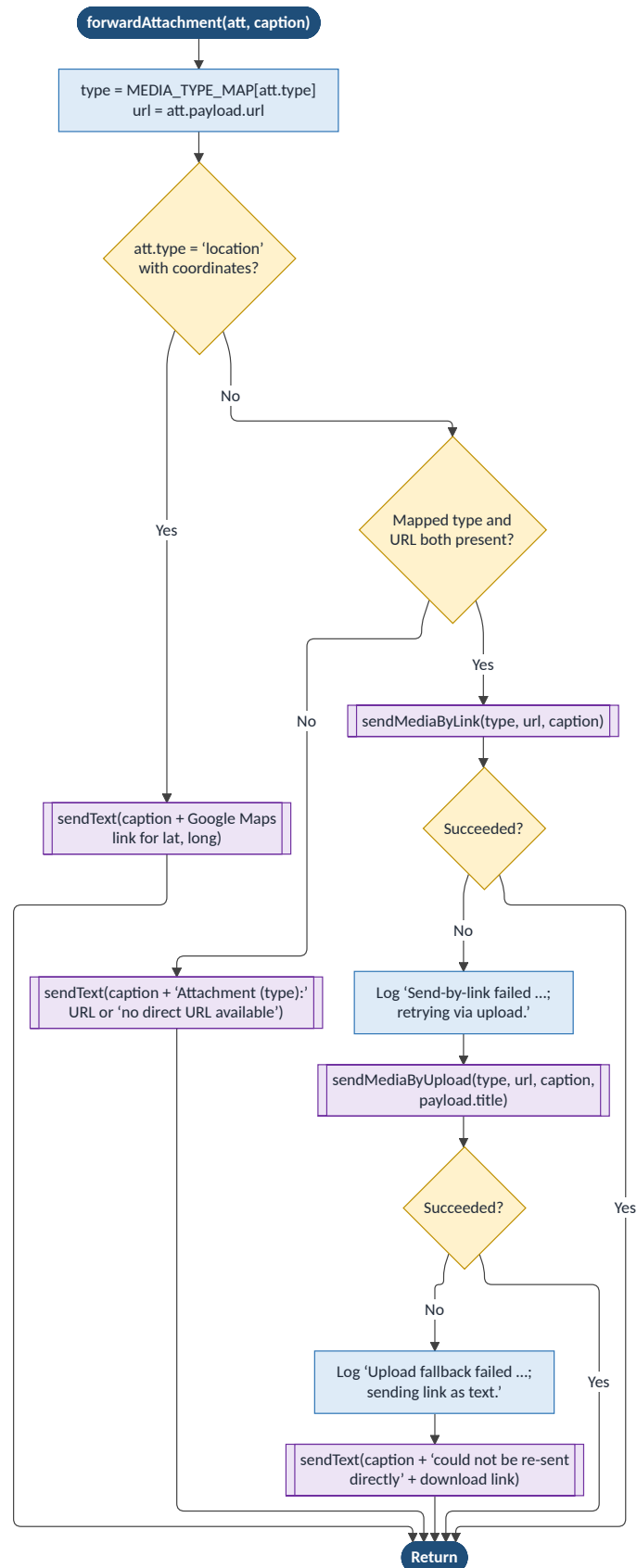
SOURCE `src/forwarder.js` – `forwardMessagingEvent()`, `markSeen()`, `header()`

Message Forwarding Logic



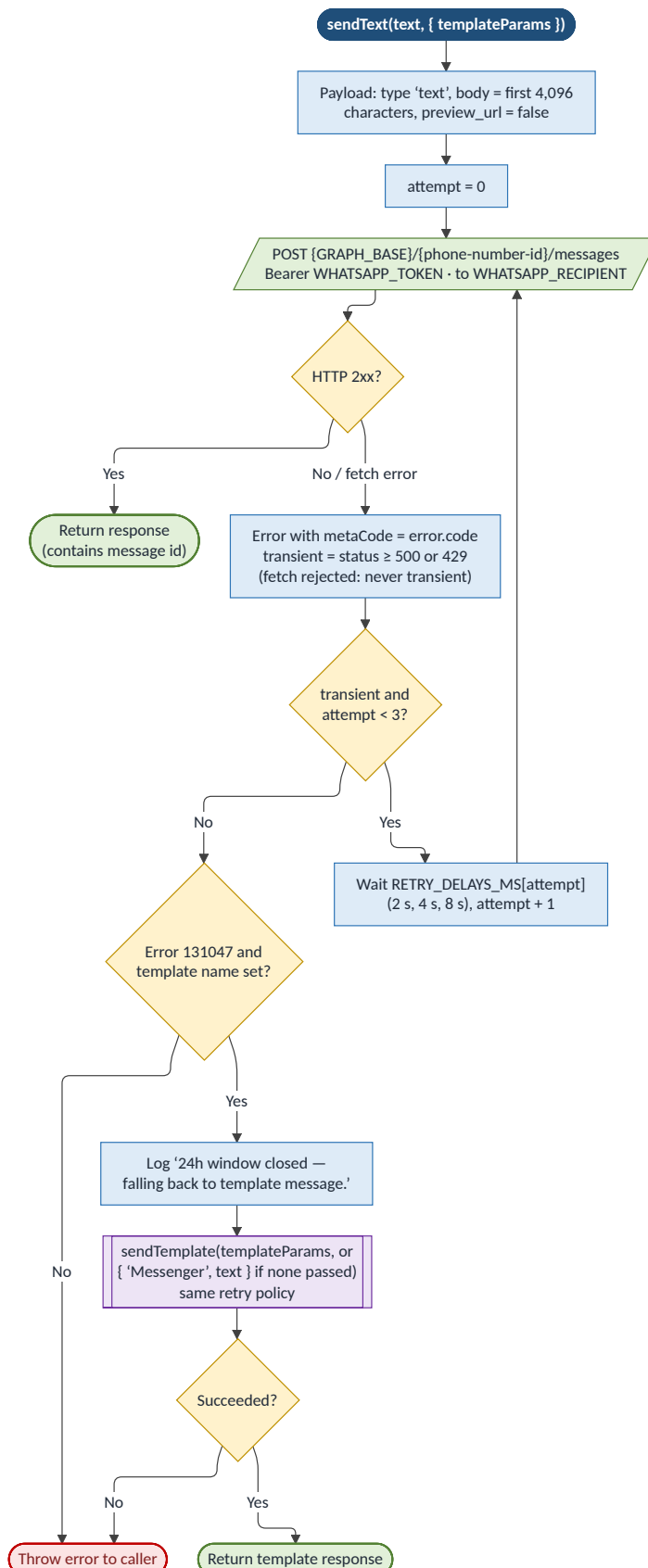
- Continues from connector A. Text is sent first; each attachment is then forwarded with the header as its caption.
- Every send is awaited in order. If one throws, the remaining sends for that event are skipped and server.js logs the error.
- The “no content” notice covers messages that have neither text nor attachments.

SOURCE src/forwarder.js – forwardMessagingEvent()



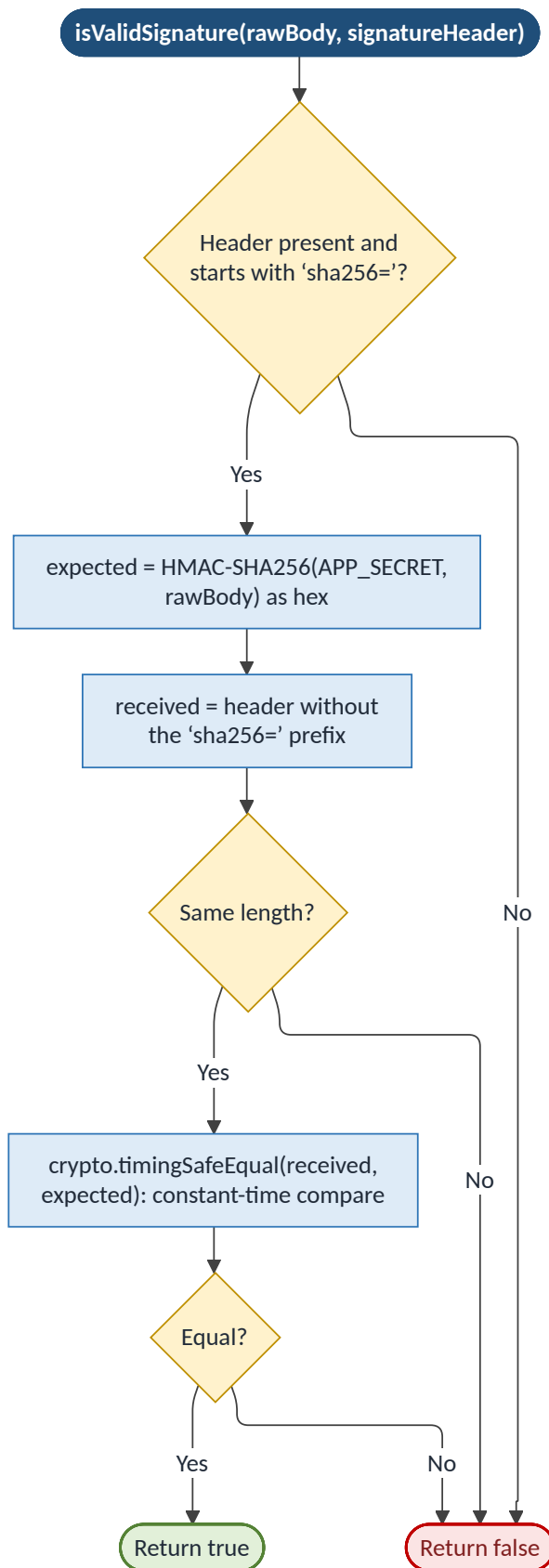
- Three-level fallback for errors the Cloud API returns on the request: send by link → download and re-upload → send the link as text.
- Captions are omitted for audio (WhatsApp does not support them) and cut to 1,024 characters. A location becomes a Google Maps link.
- Types with no WhatsApp mapping (e.g. 'fallback' link shares) are sent as a text line with their URL.

SOURCE `src/forwarder.js` – `forwardAttachment()` · `src/whatsapp.js`



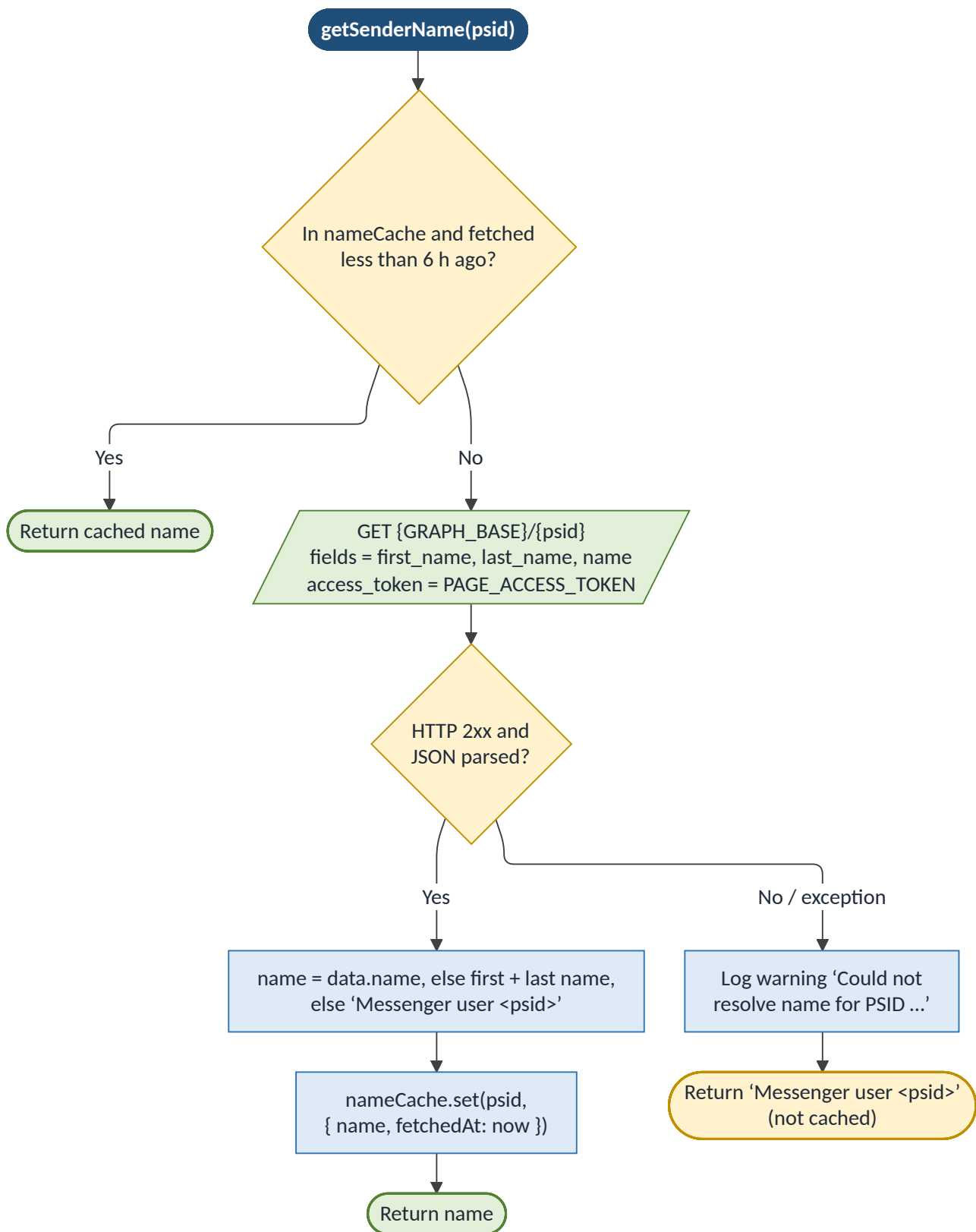
- Only transient failures (HTTP 5xx or 429) are retried: 3 retries after 2 s, 4 s and 8 s, so 4 attempts in all. A network error with no HTTP response is not retried.
- The template fallback (error 131047 and WHATSAPP_TEMPLATE_NAME set) applies to sendText only. Media sends use the same retries, then fall back as in Figure 13.
- Template parameters: {{1}} = sender name (max 200 characters), {{2}} = preview (max 500). Callers that pass none (the text fallbacks for attachments) get 'Messenger' and the message text.

SOURCE src/whatsapp.js — sendText(), sendWithRetry(), postMessages(), sendTemplate()



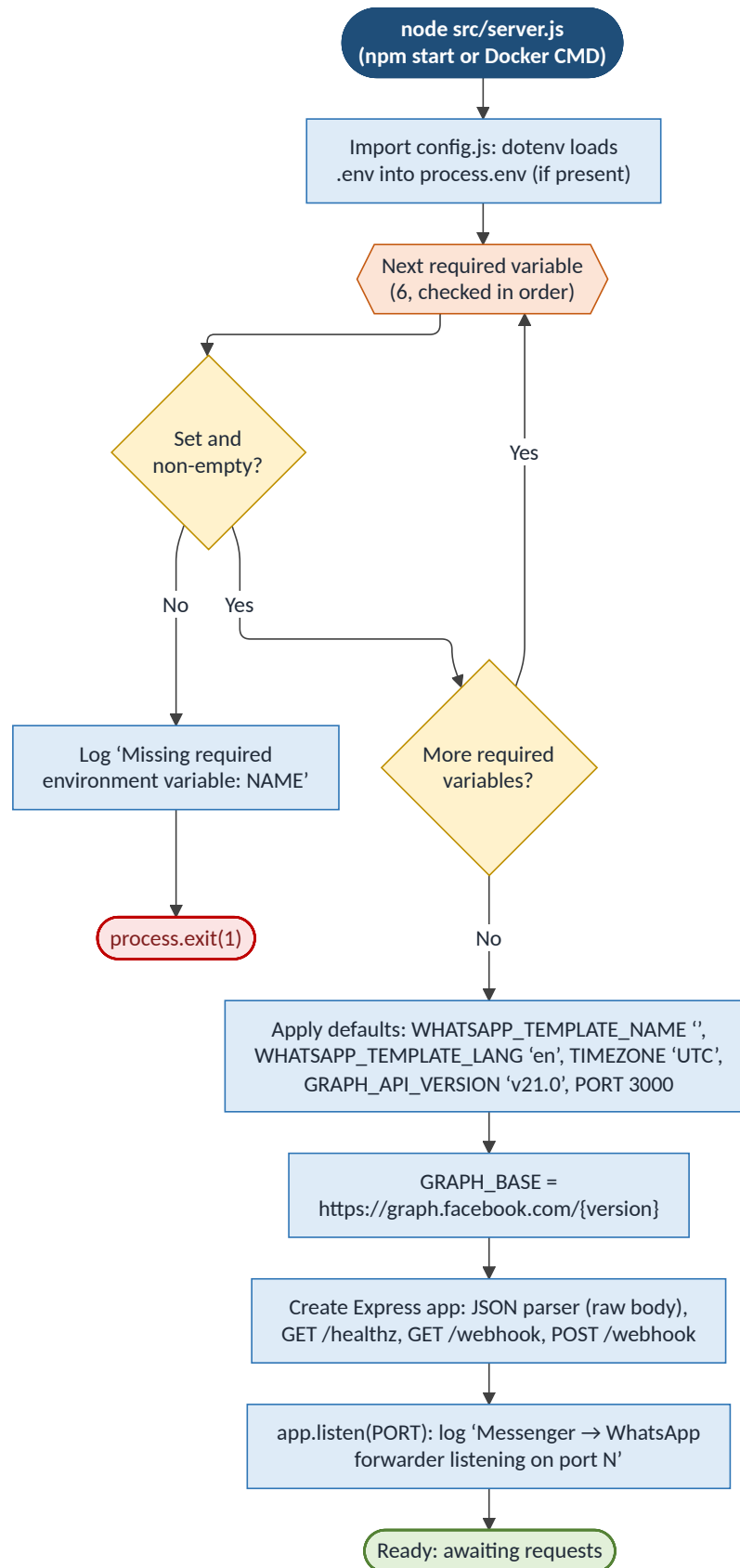
- Meta signs each webhook body with HMAC-SHA256 using the App Secret and sends it as X-Hub-Signature-256: sha256=<hex>.
- The length check prevents `crypto.timingSafeEqual` throwing on unequal lengths (it compares UTF-8 bytes, so only a non-ASCII header could still throw). The comparison is constant-time.
- Covered by four unit tests (npm test): valid signature, wrong secret, tampered body, and missing or malformed header.

SOURCE `src/verifySignature.js` · `test/verifySignature.test.js`



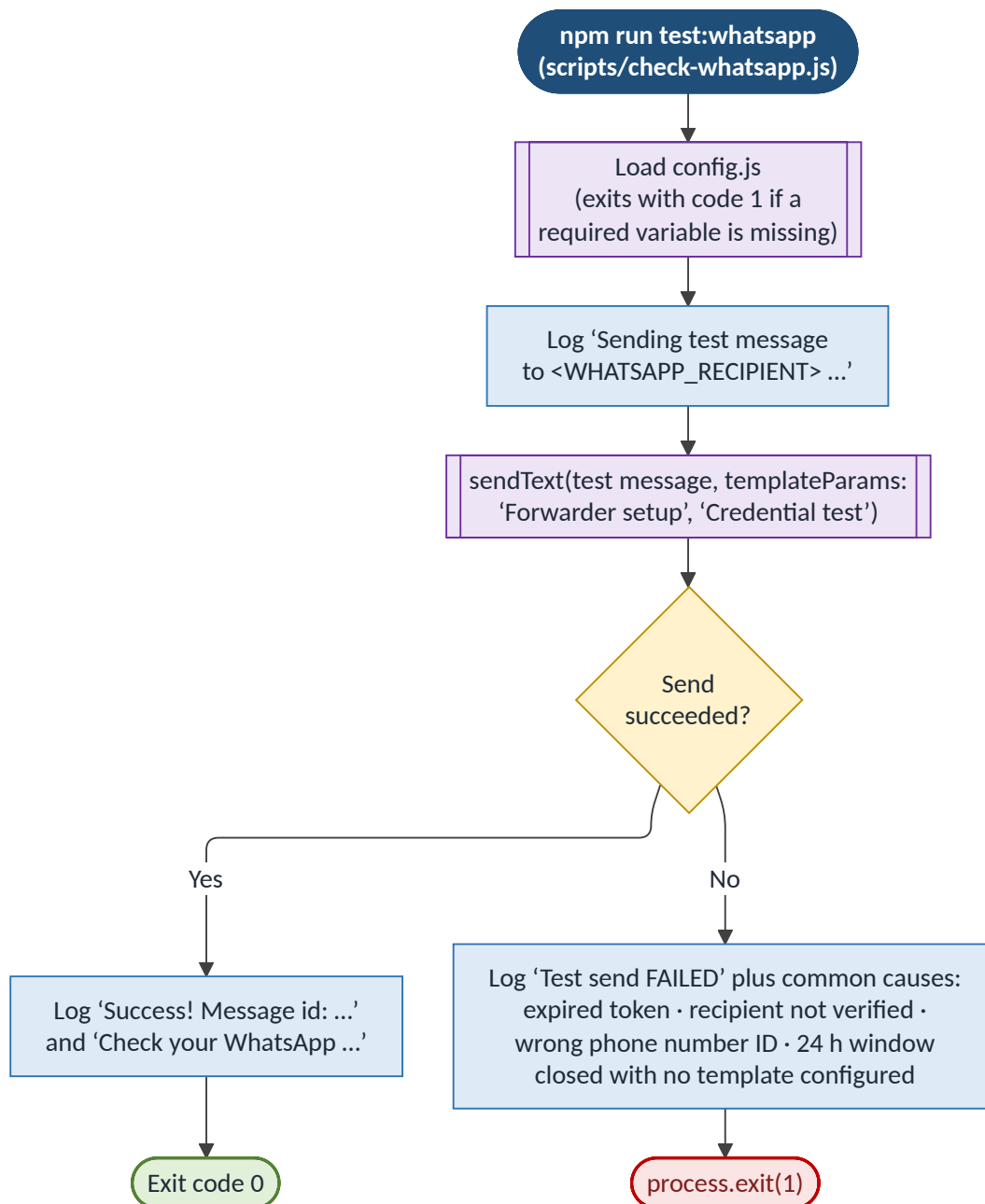
- Names are cached per PSID (page-scoped user ID) for 6 hours, so repeat messages from the same customer need no Graph API call.
- Any failure (HTTP error, network error or bad JSON) falls back to 'Messenger user <PSID>'. The fallback is not cached, so the next message tries again.
- The function never throws, so a failed lookup can never stop a message being forwarded.

SOURCE `src/messenger.js` – `getSenderName()`



- Required, in order: `APP_SECRET`, `VERIFY_TOKEN`, `PAGE_ACCESS_TOKEN`, `WHATSAPP_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_RECIPIENT`. The first missing one is logged and the process exits.
- An empty `WHATSAPP_TEMPLATE_NAME` disables the template fallback. `PORT` falls back to 3000 if unset or not a number.
- In Docker the variables come from `docker run --env-file`; running locally, `dotenv` reads the `.env` file.

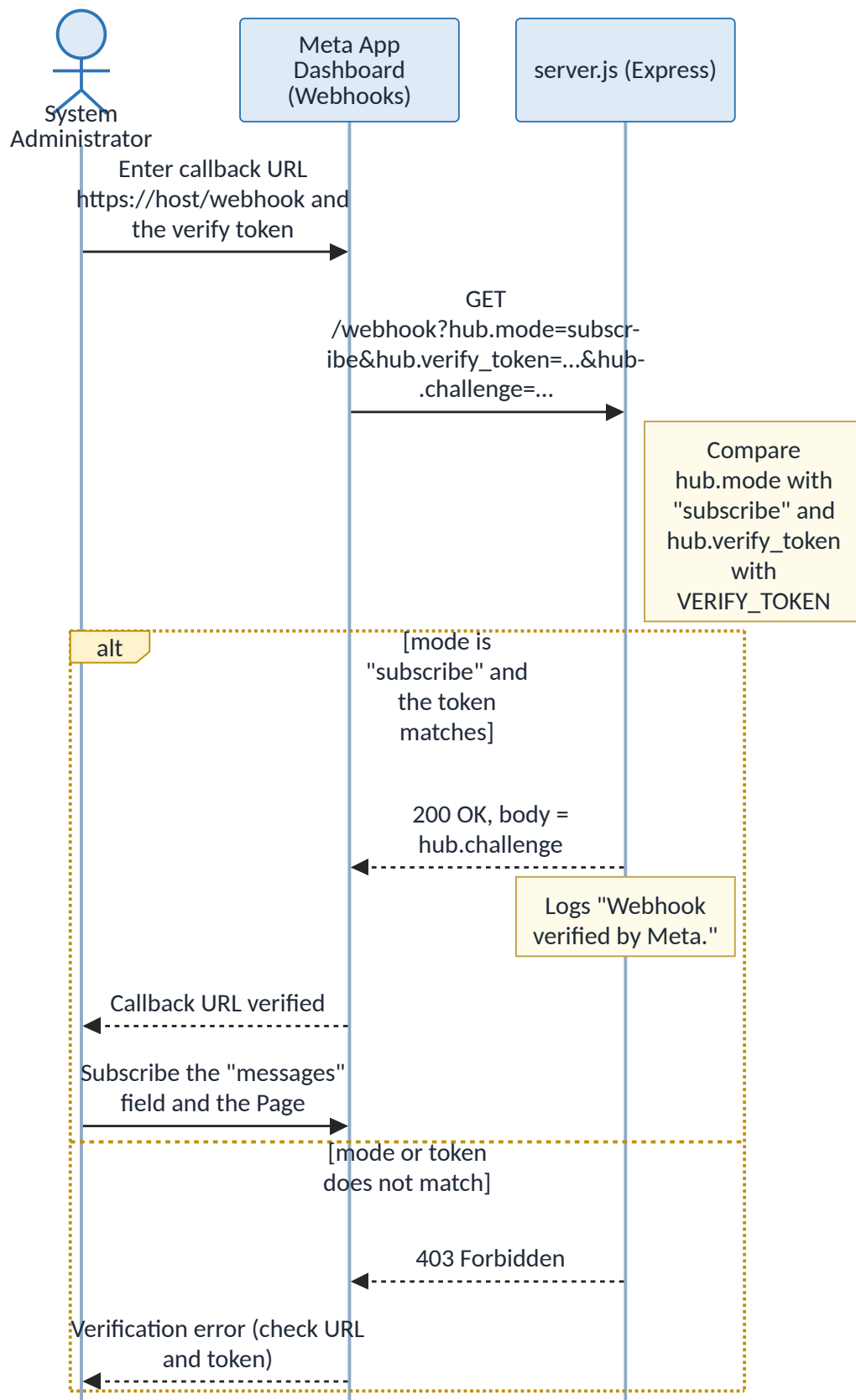
SOURCE `src/config.js` · `src/server.js`



- Run with `npm run test:whatsapp` before connecting the webhook. It sends one real message through the same `sendText()` as the service, so retries and the template fallback also apply.
- On failure it prints the four most common causes and exits with code 1. The `scripts/` folder is not copied into the Docker image, so it runs from a checkout.

SOURCE `scripts/check-whatsapp.js`

Sequence: Webhook Verification Handshake

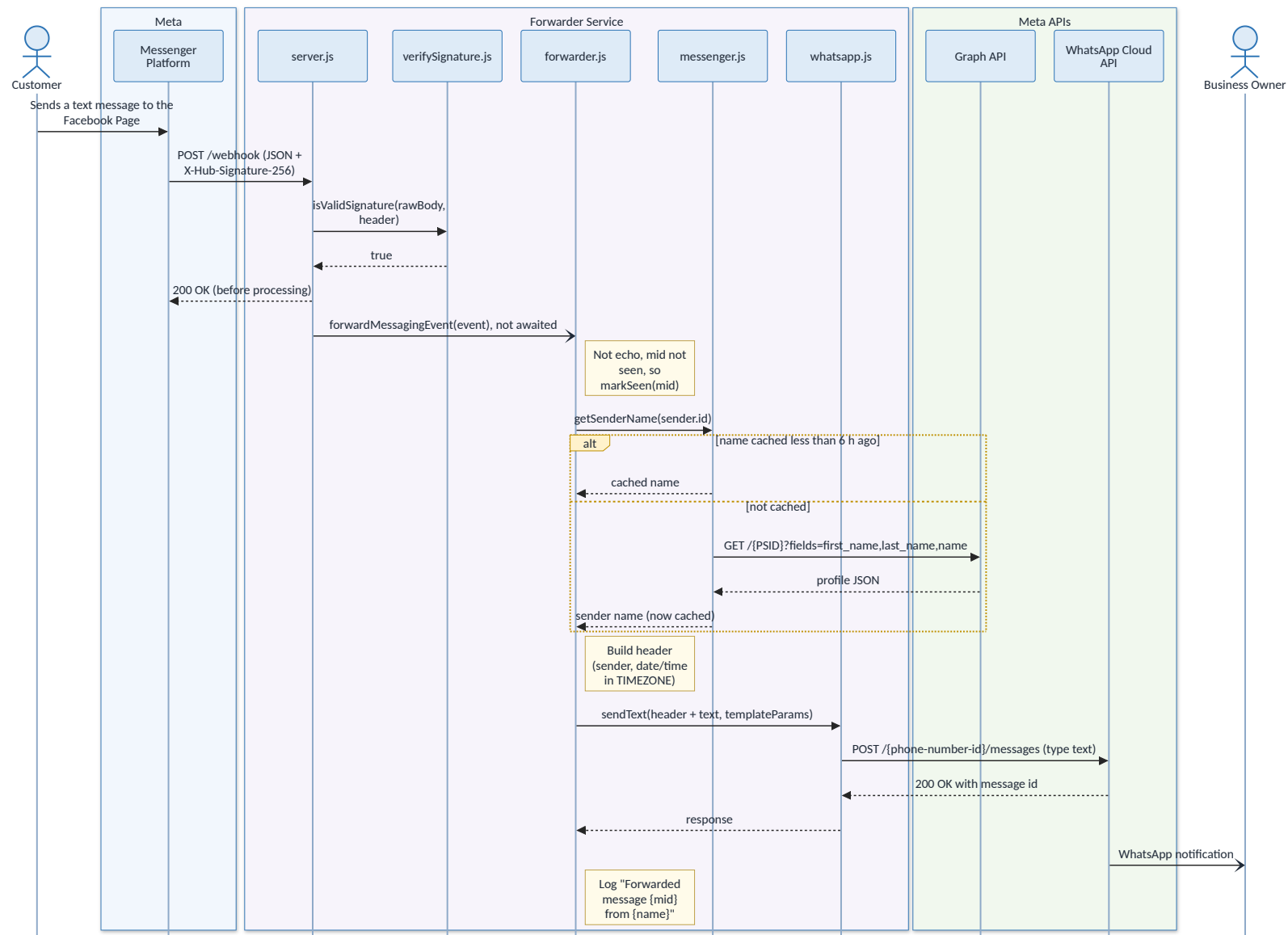


- A one-time handshake when the callback URL is saved in the Meta App Dashboard.
- The service keeps no state for this: it compares the query parameters with `VERIFY_TOKEN` and echoes `hub.challenge`.

SOURCE `src/server.js` – GET `/webhook`

FIGURE 20

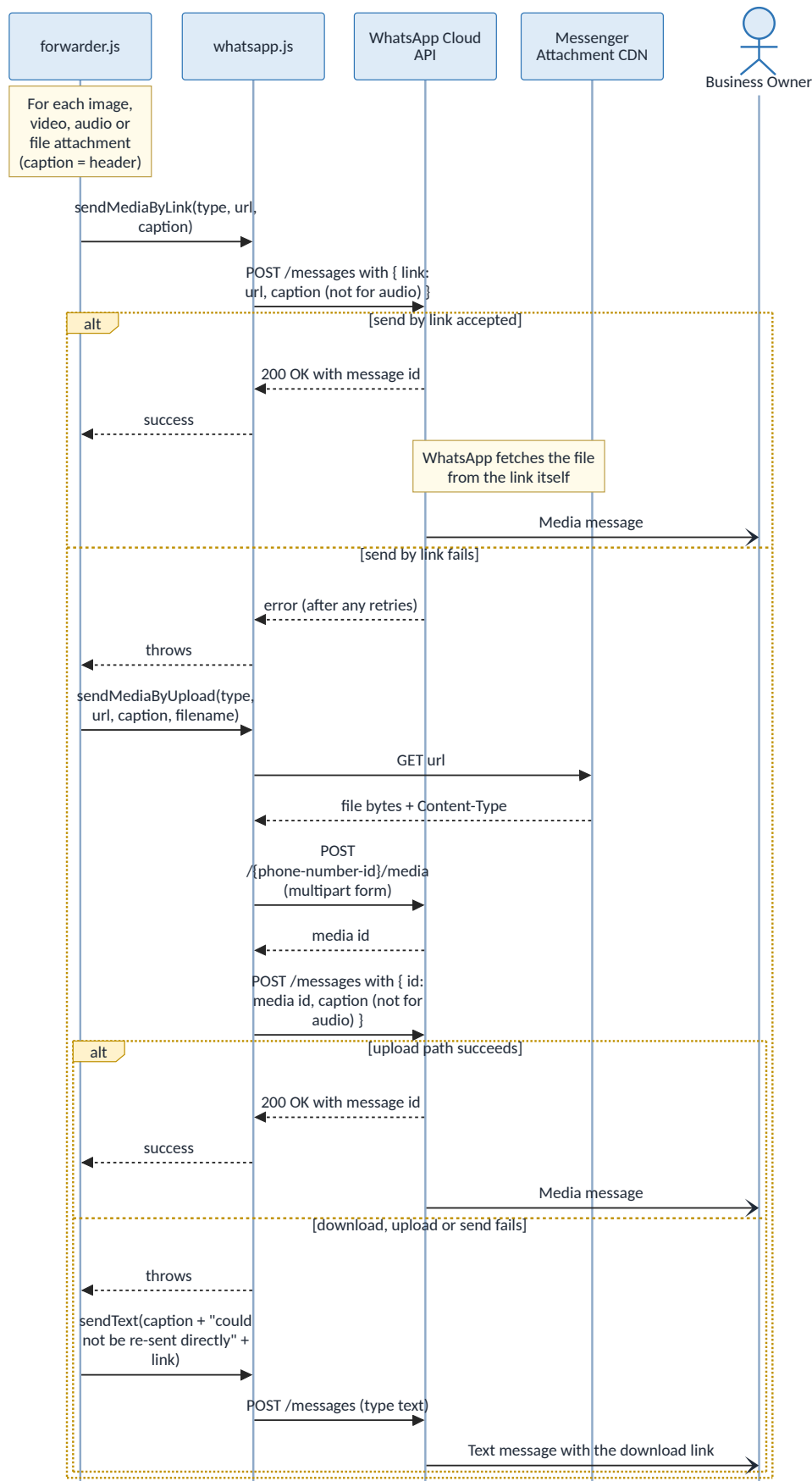
Sequence: Text Message Forwarding



- Happy path for a text message. The open arrowhead on `forwardMessagingEvent` is an asynchronous call: `server.js` has already replied 200 OK.
- The alt fragment is the 6-hour name cache: a cache hit skips the Graph API. "200 OK" from the Cloud API means accepted; WhatsApp then delivers to the phone.

SOURCE `src/server.js` · `src/forwarder.js` · `src/messenger.js` · `src/whatsapp.js`

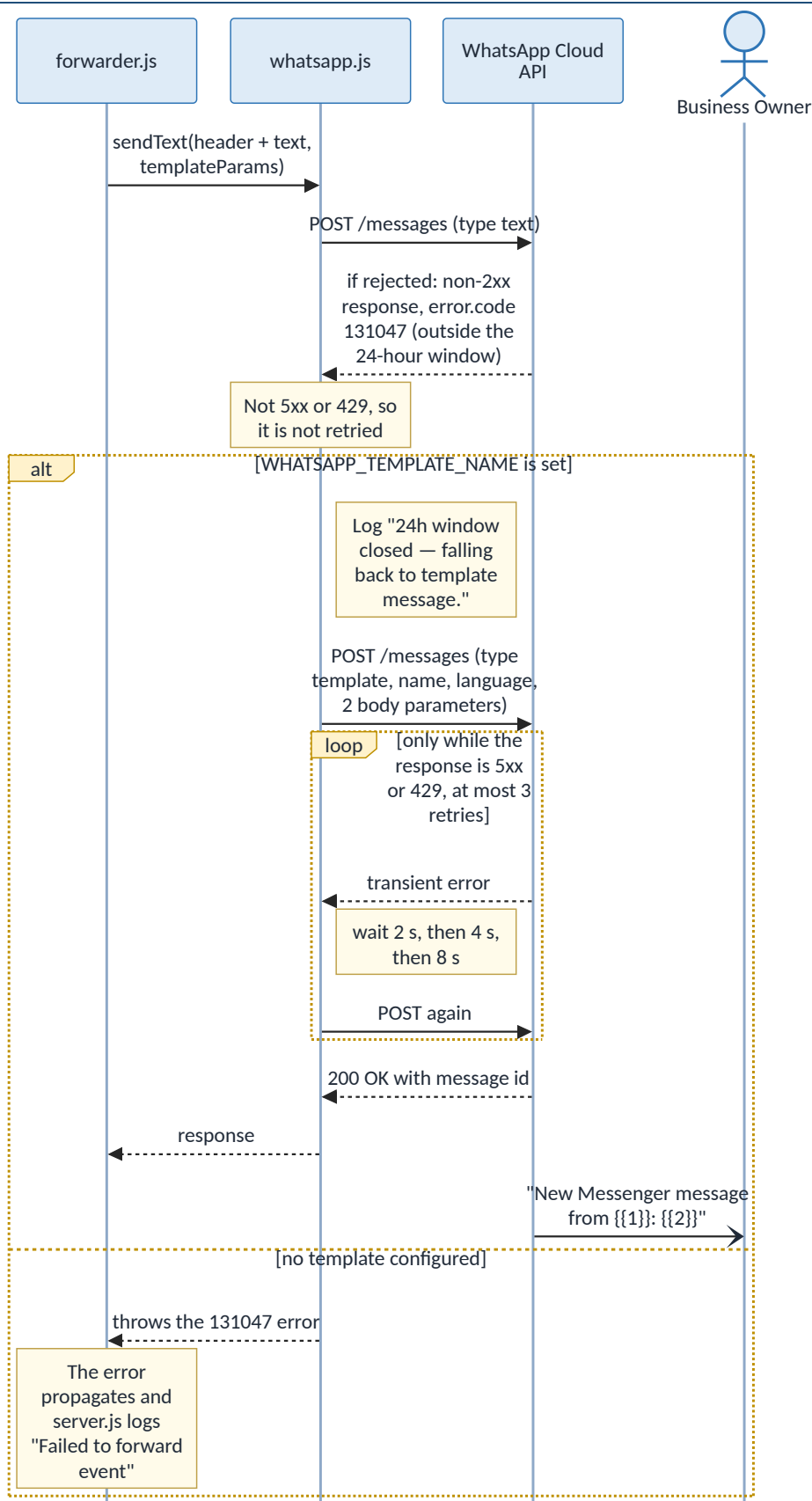
Sequence: Attachment Forwarding with Fallback



- Covers image, video, audio and file attachments (a file is sent as a WhatsApp document; audio gets no caption). Location and unmapped types take the text branches of Figure 13.
- On the send-by-link path the service passes only the URL and WhatsApp fetches the file. The upload path holds it in memory only, never on disk.

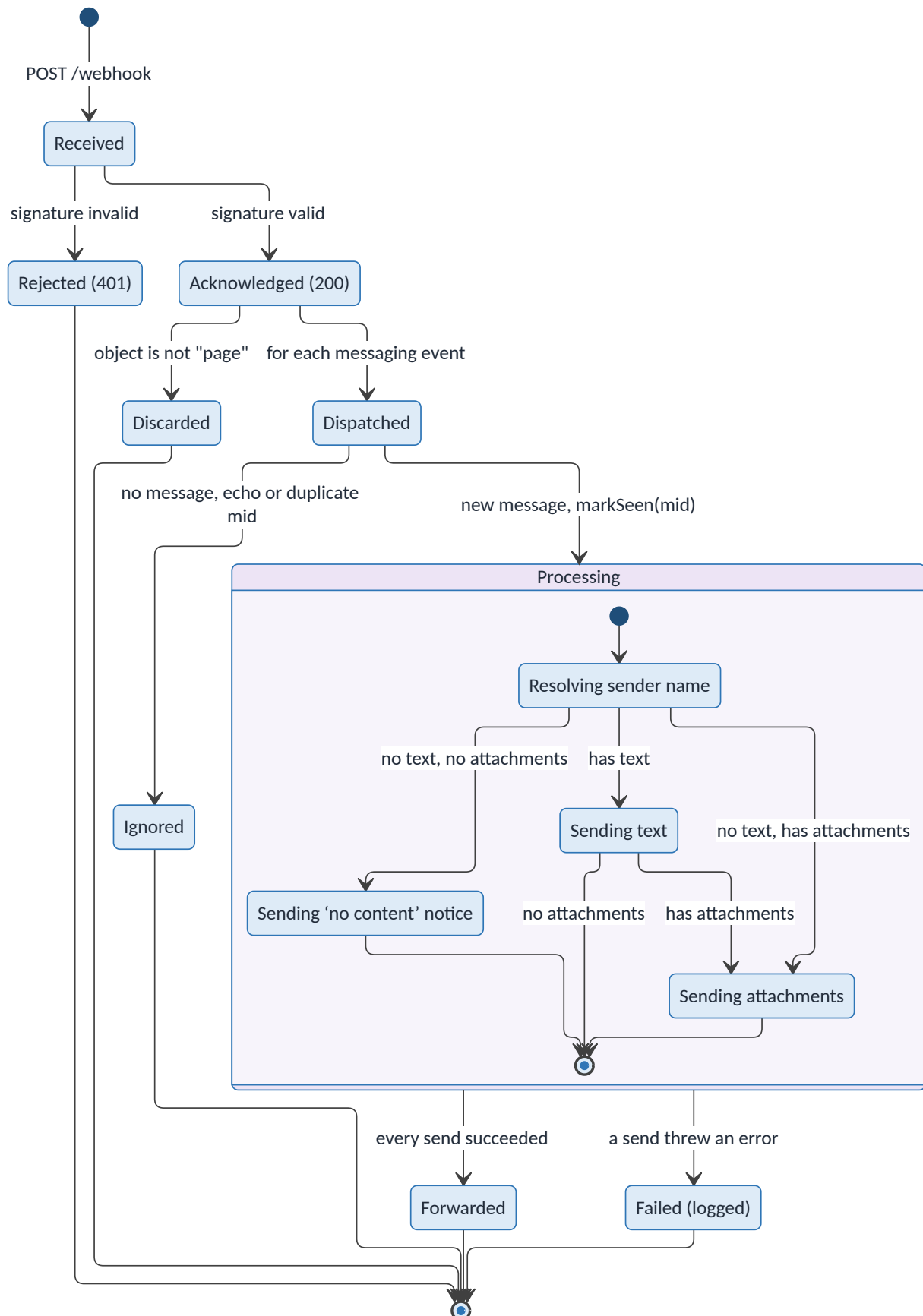
SOURCE `src/forwarder.js – forwardAttachment()` • `src/whatsapp.js – sendMediaByLink(), sendMediaByUpload()`

Sequence: 24-Hour Window and Template Fallback



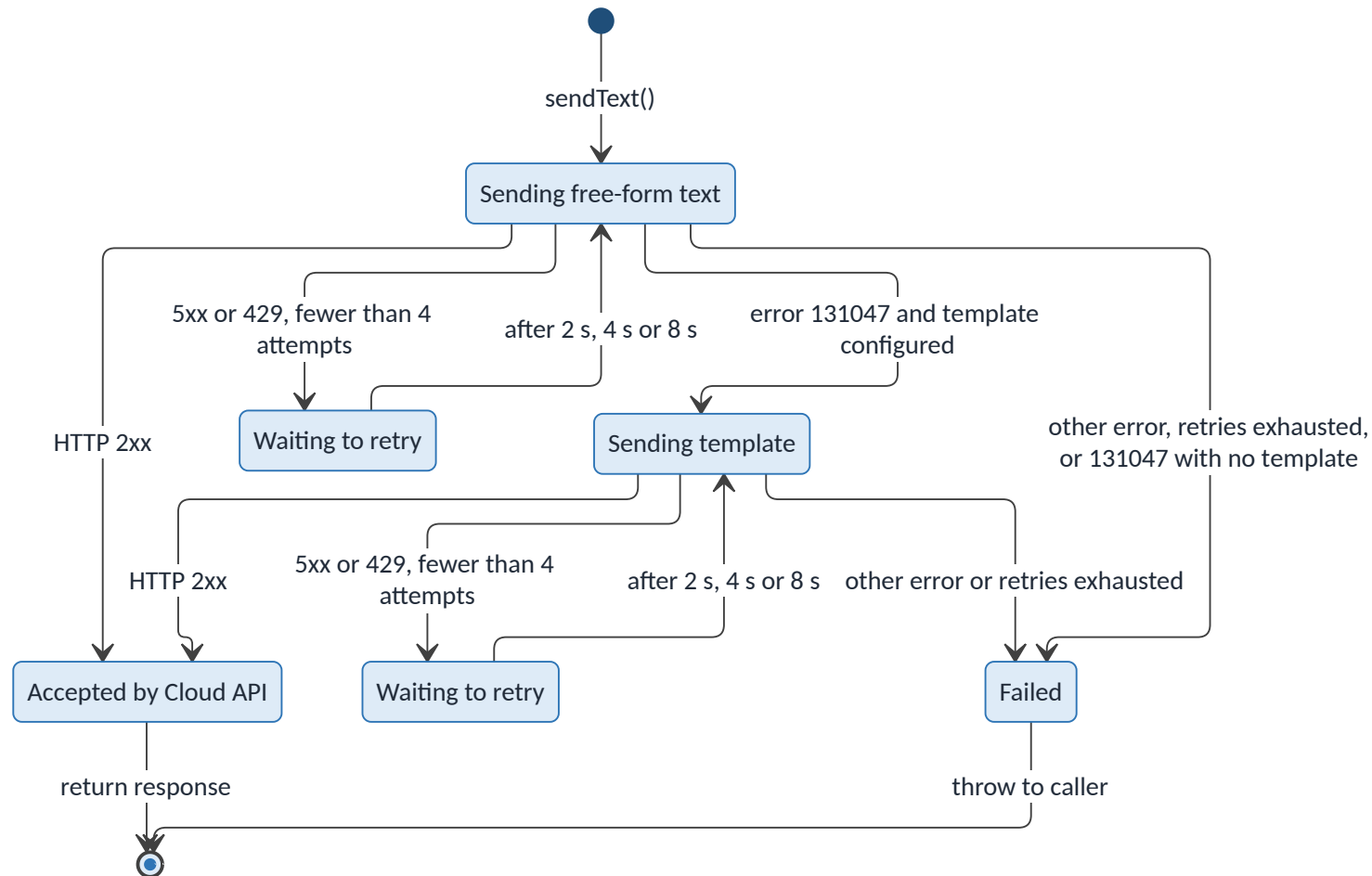
- WhatsApp accepts free-form business messages only within 24 hours of the recipient's last message; the code handles error 131047 when it is returned on the send request.
- The template must be a pre-approved Utility template with two body variables. Without `WHATSAPP_TEMPLATE_NAME` the error propagates and is logged.
- Limitation: if the Cloud API accepts the message and reports 131047 later in a status webhook, the service (which handles only object = page events) never sees it and no fallback runs.

SOURCE `src/whatsapp.js` – `sendText()`, `sendTemplate()`, `sendWithRetry()`



- Final outcomes: Rejected, Discarded, Ignored, Forwarded or Failed. For well-formed JSON requests only Rejected is visible to Meta (HTTP 401); all others follow a 200 OK.
- Failed can follow partial success, e.g. the text was delivered but an attachment's last fallback threw.

SOURCE `src/server.js` · `src/forwarder.js`



- Lifecycle of one `sendText()` call. “Accepted” means the Cloud API returned 2xx; delivery to the phone happens afterwards, inside WhatsApp.
- The free-form send and the template send each have their own retry budget: up to 4 attempts (1 + 3 retries) for transient errors.

SOURCE `src/whatsapp.js`